

# React main.jsx Complete Guide

---

**main.jsx** হলো একটি React অ্যাপ্লিকেশনের **Entry Point**। React অ্যাপ চালু হওয়ার সময় সবার আগে এই ফাইলটি Execute হয়। এখান থেকেই React Application Browser-এ Render হয়।

## Complete Code

```
import React from 'react'
import { createRoot } from 'react-dom/client'
import { RouterProvider } from 'react-router-dom'
import './index.css'
import router from './routes/routes'

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>,
)
```

## Line 1

```
import React from 'react'
```

## Explanation

**React** লাইব্রেরিটি Import করা হচ্ছে।

React ব্যবহার করে আমরা—

- Component তৈরি করি
- JSX লিখি
- Hooks ব্যবহার করি
- Virtual DOM ব্যবহার করি

React হলো UI তৈরির জন্য JavaScript Library।

💡 **Note:** React 17+ এ অনেক ক্ষেত্রে এই Import না দিলেও চলে, কিন্তু অনেক Project-এ এখনও রাখা হয়।

## Line 2

```
import { createRoot } from 'react-dom/client'
```

### Explanation

এখানে `createRoot()` Import করা হচ্ছে।

এর কাজ হলো React Application-কে Browser-এর HTML এর সাথে যুক্ত করা।

React নিজে UI তৈরি করে, আর **react-dom** সেই UI Browser-এ দেখায়।

আগে React 17 পর্যন্ত ব্যবহার হতো—

```
ReactDOM.render(<App />, document.getElementById("root"))
```

React 18 থেকে ব্যবহার হয়—

```
createRoot(document.getElementById("root"))
```

এটি নতুন Rendering API।

## Line 3

```
import { RouterProvider } from 'react-router-dom'
```

## Explanation

**RouterProvider** React Router-এর একটি Component।

এটি পুরো Application-এ Routing Enable করে।

এর ফলে—

- `/`
- `/about`
- `/contact`
- `/dashboard`

এসব Route কাজ করে।

যদি **RouterProvider** না থাকে তাহলে—

- Link কাজ করবে না
- `useNavigate()` কাজ করবে না
- `useLocation()` কাজ করবে না
- URL Change হলেও Page Change হবে না

## Line 4

```
import './index.css'
```

## Explanation

এটি Global CSS File।

এখানে লেখা Style পুরো Project-এ কাজ করবে।

যেমন—

```
body{  
  margin:0;  
  font-family:sans-serif;  
}
```

## Line 5

```
import router from './routes/routes'
```

## Explanation

এখানে Routes Configuration Import করা হচ্ছে।

উদাহরণ—

```
const router = createBrowserRouter([  
  {  
    path: "/",  
    element: <Home />  
  },  
  {  
    path: "/about",  
    element: <About />  
  }  
])
```

এই Router-টাই পরে **RouterProvider** -কে দেওয়া হবে।

## Line 6

```
createRoot(document.getElementById('root'))
```

## Explanation

প্রথমে HTML থেকে

```
<div id="root"></div>
```

Element-টি খুঁজে বের করা হয়।

তারপর React সেই Element-কে Root হিসেবে ব্যবহার করে।

এই Root-এর ভিতরেই পুরো React Application Render হবে।

## Line 7

```
.render(
```

## Explanation

`render()` এর কাজ হলো React Component Browser-এ দেখানো।

React Application এখান থেকেই Screen-এ Display হতে শুরু করে।

## Line 8

```
<React.StrictMode>
```

## What is StrictMode?

`StrictMode` হলো React-এর একটি Development Tool।

এটি User Interface-এ কিছু দেখায় না।

শুধুমাত্র Development Mode-এ অতিরিক্ত Checking করে।

## কাজ

Deprecated API Detect করে

Memory Leak ধরতে সাহায্য করে

Side Effect Detect করে

Future Compatibility নিশ্চিত করে

Best Practice Follow করতে সাহায্য করে

*Production Build-এ StrictMode কোনো Effect ফেলে না।*

## Line 9

```
<RouterProvider router={router} />
```

## Explanation

এখানে Router Configuration React Application-কে দেওয়া হচ্ছে।

এখন React জানে—

যদি URL হয়

```
/
```

তাহলে Home Component দেখাও।

যদি

```
/about
```

তাহলে About Component দেখাও।

যদি

`/dashboard`

তাহলে Dashboard Component দেখাও।

এটাই Routing System-এর মূল কাজ।

## Complete Flow

Browser

|

▼

`index.html`

|

▼

`<div id="root"></div>`

|

▼

`main.jsx`

|

▼

Import React

|

▼

Import CSS

|

▼

Import Router

|

▼

Create React Root

|

▼

StrictMode

|

▼

RouterProvider

|

▼

React Application Start

## Interview Questions

---

1. What is `main.jsx` ?
2. Why do we use `createRoot()` ?
3. What is the purpose of `React.StrictMode` ?
4. What is `RouterProvider` ?
5. What happens if we remove `RouterProvider` from `main.jsx` ?

## Key Points

---



- **main.jsx** হলো React Application-এর Entry Point।
- **createRoot()** Browser-এর Root Element-এর সাথে React-কে Connect করে।
- **StrictMode** Development-এর সময় Bug Detect করতে সাহায্য করে।
- **RouterProvider** পুরো Application-এ Routing Enable করে।
- **index.css** Global Styling-এর জন্য ব্যবহার করা হয়।

## React Router ( **routes.jsx** ) Complete Guide

---

**routes.jsx** ফাইলটি React Router-এর **Routing Configuration File**।

এই ফাইলের মাধ্যমে আমরা নির্ধারণ করি কোন **URL (Route)**-এ কোন **Component (Page)** দেখানো হবে।

সহজ ভাষায়,

*User Browser-এ যে URL লিখবে, React Router সেই অনুযায়ী Component Render করবে।*

## Complete Code

---

```
import { createBrowserRouter } from 'react-router-dom'
import Mainlayout from '../Layout/Mainlayout'
import Home from '../pages/home'
import About from '../pages/about'

const router = createBrowserRouter([
  {
    path: '/',
    element: <Mainlayout />,
    children: [
      {
        path: '/',
        element: <Home />
      },
      {
        path: '/about',
        element: <About />
      }
    ]
  }
])

export default router
```

## Line 1

---

```
import { createBrowserRouter } from 'react-router-dom'
```

## Explanation

এখানে `createBrowserRouter()` Import করা হচ্ছে।

এটি React Router-এর একটি Function।

এর কাজ হলো

- Application-এর সব Route তৈরি করা
- URL অনুযায়ী Component দেখানো
- Navigation Handle করা

## উদাহরণ

/

↓

Home Page

/about

↓

About Page

/contact

↓

Contact Page

## Line 2

---

```
import Mainlayout from '../Layout/Mainlayout'
```

## Explanation

এখানে Main Layout Import করা হয়েছে।

Main Layout এমন একটি Component যেখানে সাধারণত থাকে

- Navbar
- Footer

- Outlet
- Sidebar (প্রয়োজনে)

একই Layout-এর ভিতরে অনেক Page দেখানো যায়।

উদাহরণ

Navbar

↓

Current Page

↓

Footer

## Line 3

---

```
import Home from '../pages/home'
```

## Explanation

Home Page Component Import করা হচ্ছে।

যখন User

/

Route-এ যাবে,

তখন Home Component দেখানো হবে।

## Line 4

---

```
import About from '../pages/about'
```

## Explanation

About Page Import করা হচ্ছে।

যখন User

```
/about
```

Route-এ যাবে,

তখন About Component Render হবে।

## Line 5

---

```
const router = createBrowserRouter([
```

## Explanation

এখানে Router Object তৈরি করা হচ্ছে।

এই Router-এর ভিতরে Application-এর সব Route লেখা হয়।

প্রতিটি Route একটি JavaScript Object।

## প্রথম Route Object

---

```
{  
  path: '/',  
  element:<Mainlayout/>  
}
```

## path

```
path: '/'
```

**path** মানে URL।

```
/
```

মানে Website-এর Home URL।

## element

```
element:<Mainlayout/>
```

**element** মানে কোন Component Render হবে।

এখানে MainLayout Render হবে।

## children

---

```
children:[]
```

## children কী?

Children হলো Nested Routes।

অর্থাৎ MainLayout-এর ভিতরে কোন কোন Page দেখানো হবে তা এখানে লেখা হয়।

MainLayout সাধারণত এমন হয়—

```

<>
  <Navbar />

  <Outlet />

  <Footer />
</>

```

`<Outlet />` যেখানে থাকবে, সেখানে Children Route-এর Component Render হবে।

## First Child

```

{
  path: '/',
  element: <Home/>
}

```

যখন User যাবে

```

/

```

React Render করবে

```

MainLayout
↓
Home

```

Browser-এ দেখা যাবে

```

Navbar

Home Page

```

Footer

## Second Child

---

```
{
  path: '/about',
  element: <About/>
}
```

যখন User যাবে

/about

React Render করবে

MainLayout

↓

About

Browser-এ দেখা যাবে

Navbar

About Page

Footer

## Flow Diagram

---



Browser URL

↓

createBrowserRouter()

↓

Match Route

↓

MainLayout

↓

Outlet

↓

Child Component

↓

Browser Screen

## আরও Route কীভাবে Add করবেন

---

### Contact Page

প্রথমে Component Import করুন।

```
import Contact from '../pages/contact'
```

তারপর children-এর ভিতরে লিখুন

```
{  
  path: '/contact',
```

```
    element:<Contact/>
  }
```

এখন

```
/contact
```

লিখলে Contact Page দেখা যাবে।

## Service Page

```
import Service from '../pages/service'
```

```
{
  path: '/service',
  element:<Service/>
}
```

## Blog Page

```
import Blog from '../pages/blog'
```

```
{
  path: '/blog',
  element:<Blog/>
}
```

## Dashboard Page

```
import Dashboard from '../pages/dashboard'
```

```
{  
  path: '/dashboard',  
  element: <Dashboard/>  
}
```

## Multiple Children Example

---

```
children:[  
  {  
    path: '/',  
    element: <Home/>  
  },  
  {  
    path: '/about',  
    element: <About/>  
  },  
  {  
    path: '/contact',  
    element: <Contact/>  
  },  
  {  
    path: '/blog',  
    element: <Blog/>  
  },  
  {  
    path: '/service',  
    element: <Service/>  
  },  
  {  
    path: '/dashboard',  
    element: <Dashboard/>  
  }  
]
```

# Multiple Parent Layout Example

React Router-এ একাধিক Parent Layout ব্যবহার করা যায়।

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <Mainlayout />,
    children: [
      {
        path: '/',
        element: <Home />
      },
      {
        path: '/about',
        element: <About />
      }
    ]
  },
  {
    path: '/dashboard',
    element: <DashboardLayout />,
    children: [
      {
        path: '/dashboard',
        element: <DashboardHome />
      },
      {
        path: '/dashboard/profile',
        element: <Profile />
      },
      {
        path: '/dashboard/settings',
        element: <Settings />
      }
    ]
  }
])
```

এখানে

- `/` দিয়ে শুরু হওয়া Route-গুলো **MainLayout** ব্যবহার করবে।
- `/dashboard` দিয়ে শুরু হওয়া Route-গুলো **DashboardLayout** ব্যবহার করবে।

# Route Object-এর গুরুত্বপূর্ণ Property

| Property                  | কাজ                                  |
|---------------------------|--------------------------------------|
| <code>path</code>         | URL নির্ধারণ করে                     |
| <code>element</code>      | কোন Component Render হবে             |
| <code>children</code>     | Nested Route তৈরি করে                |
| <code>loader</code>       | Route Load হওয়ার আগে Data Fetch করে |
| <code>action</code>       | Form Submit Handle করে               |
| <code>errorElement</code> | Error হলে কোন Component দেখাবে       |
| <code>lazy</code>         | Lazy Loading-এর জন্য ব্যবহার হয়     |

## Interview Questions

### 1. What is `createBrowserRouter()` ?

এটি React Router-এর একটি Function যা পুরো Application-এর Routing Configuration তৈরি করে।

### 2. What is the purpose of `path` ?

`path` URL নির্ধারণ করে। User যে URL-এ যাবে, সেই Route Match হবে।

### 3. What is the purpose of `element` ?

`element` নির্ধারণ করে কোন React Component Render হবে।

### 4. What is `children` in React Router?

`children` Nested Route তৈরি করতে ব্যবহৃত হয়। Parent Layout-এর `<Outlet />` -এর ভিতরে Child Component Render হয়।

### 5. Why do we use `MainLayout` ?

একই Navbar, Footer, Sidebar ইত্যাদি একাধিক Page-এ পুনরায় না লিখে Reuse করার জন্য `MainLayout` ব্যবহার করা হয়।

## Key Points

- `createBrowserRouter()` দিয়ে Router তৈরি করা হয়।
- `path` URL নির্ধারণ করে।
- `element` কোন Component দেখাবে তা নির্ধারণ করে।
- `children` Nested Route তৈরি করে।
- Parent Layout-এর `<Outlet />` -এ Child Route Render হয়।
- নতুন Route যোগ করতে Component Import করে `children` -এ নতুন Object যোগ করতে হয়।
- বড় Project-এ একাধিক Layout ব্যবহার করা যায় (যেমন `MainLayout` , `DashboardLayout` )।

## React MainLayout এবং `<Outlet />` Complete Guide

---

`MainLayout` হলো একটি **Parent Layout Component**।

এর কাজ হলো এমন সব Component এক জায়গায় রাখা যেগুলো Website-এর প্রতিটি Page-এ একই থাকবে।

যেমন—

- Navbar
- Footer
- Sidebar
- Header
- Theme Provider
- Notification
- Scroll To Top

এগুলো প্রতিটি Page-এ আলাদা আলাদা লিখতে হয় না।

### Example MainLayout

---

```
import { Outlet } from 'react-router-dom'
import Navbar from '../components/Navbar'
import Footer from '../components/Footer'

const MainLayout = () => {
```

```
return (  
  <>  
    <Navbar />  
  
    <Outlet />  
  
    <Footer />  
  </>  
)  
}  
  
export default MainLayout
```

## কেন MainLayout ব্যবহার করি?

---

ধরুন আপনার Website-এ ১০টি Page আছে।

```
Home  
  
About  
  
Contact  
  
Blog  
  
Services  
  
Gallery  
  
Dashboard  
  
Profile  
  
Settings  
  
FAQ
```

যদি MainLayout ব্যবহার না করেন,  
তাহলে প্রতিটি Page-এ লিখতে হবে—

```
<>
  <Navbar />

  Home Page

  <Footer />
</>
```

তারপর আবার About Page-এ

```
<>
  <Navbar />

  About Page

  <Footer />
</>
```

আবার Contact Page-এ

```
<>
  <Navbar />

  Contact Page

  <Footer />
</>
```

এভাবে প্রতিটি Page-এ একই Code বারবার লিখতে হবে।

এটাকে বলে **Code Duplication**।

## MainLayout ব্যবহার করলে

---

Navbar এবং Footer একবারই লিখতে হবে।



```
<>
  <Navbar />

  <Outlet />

  <Footer />
</>
```

এখন শুধু Outlet-এর ভিতরে Page পরিবর্তন হবে।

## <Outlet /> কী?

<Outlet /> হলো React Router-এর একটি Component।

এর কাজ হলো—

*Child Route যেখানে Render হবে সেই জায়গা নির্ধারণ করা।*

সহজ ভাষায়,

Outlet হচ্ছে একটি **Placeholder**।

## উদাহরণ

ধরুন MainLayout হলো

```
<>
  <Navbar />

  <Outlet />

  <Footer />
</>
```

এবং Route হলো

```
children:[
  {
```

```
    path: '/',  
    element: <Home/>  
  },  
  {  
    path: '/about',  
    element: <About/>  
  }  
]
```

## User যদি যায়

/

তাহলে Browser-এ হবে

Navbar

↓

Home

↓

Footer

## User যদি যায়

/about

তাহলে হবে

Navbar



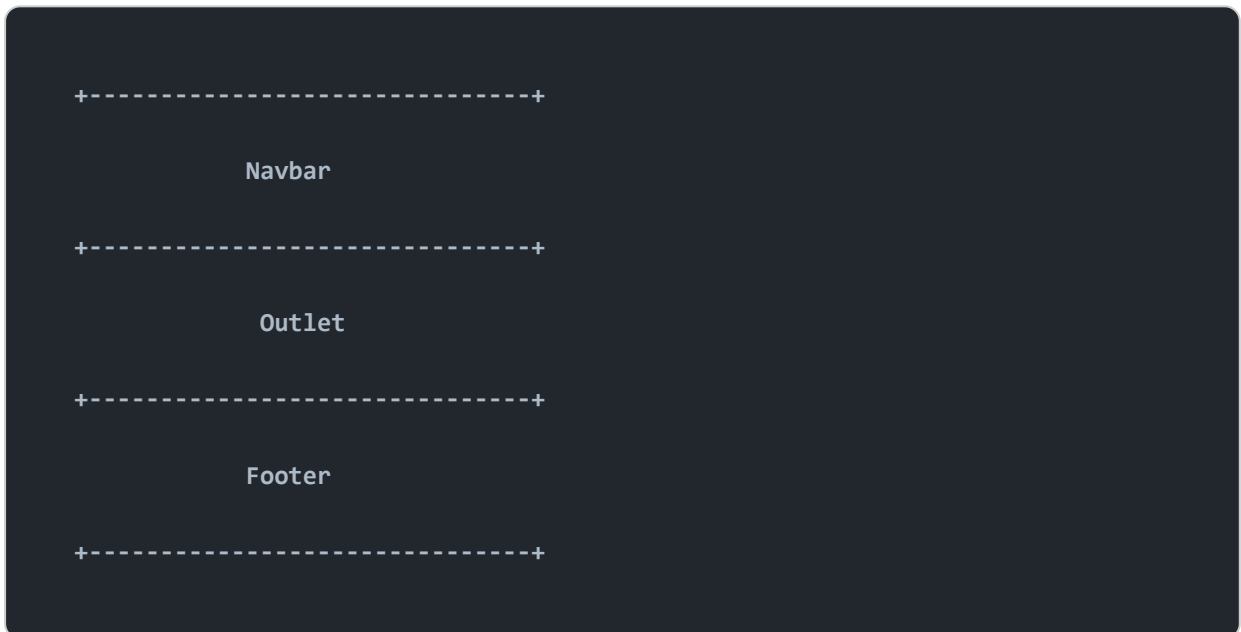
দেখুন,

Navbar এবং Footer একই থাকছে।

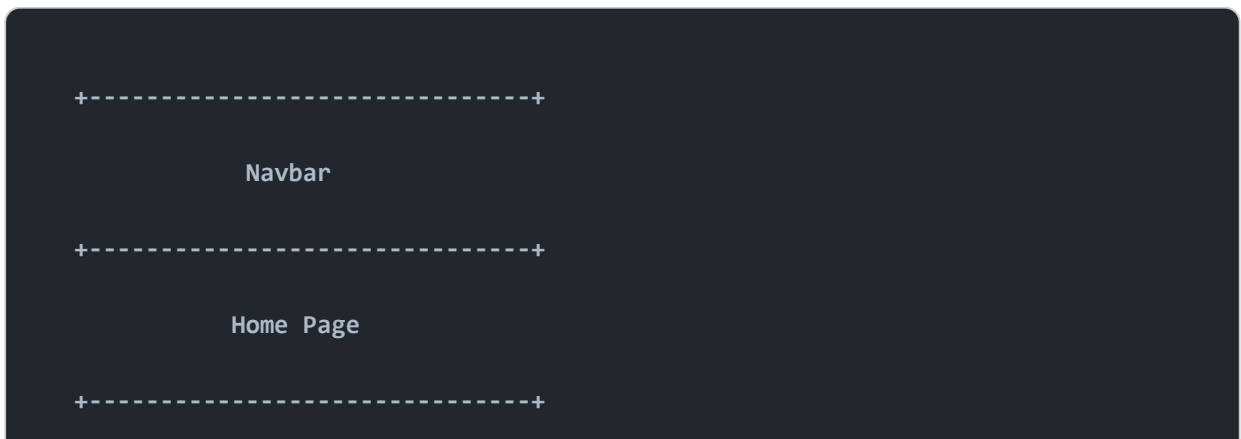
শুধু Outlet-এর ভিতরের Component পরিবর্তন হচ্ছে।

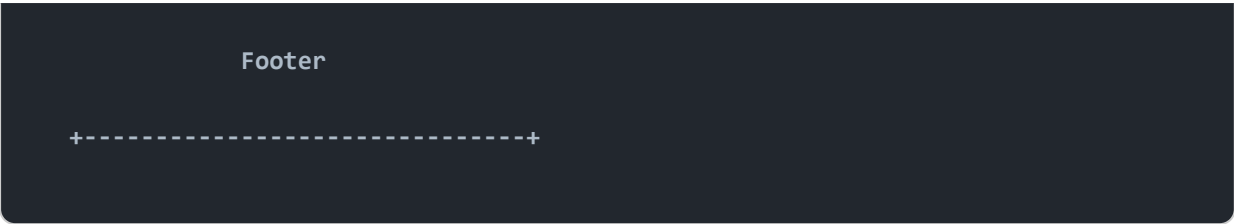
## Screen Diagram

---

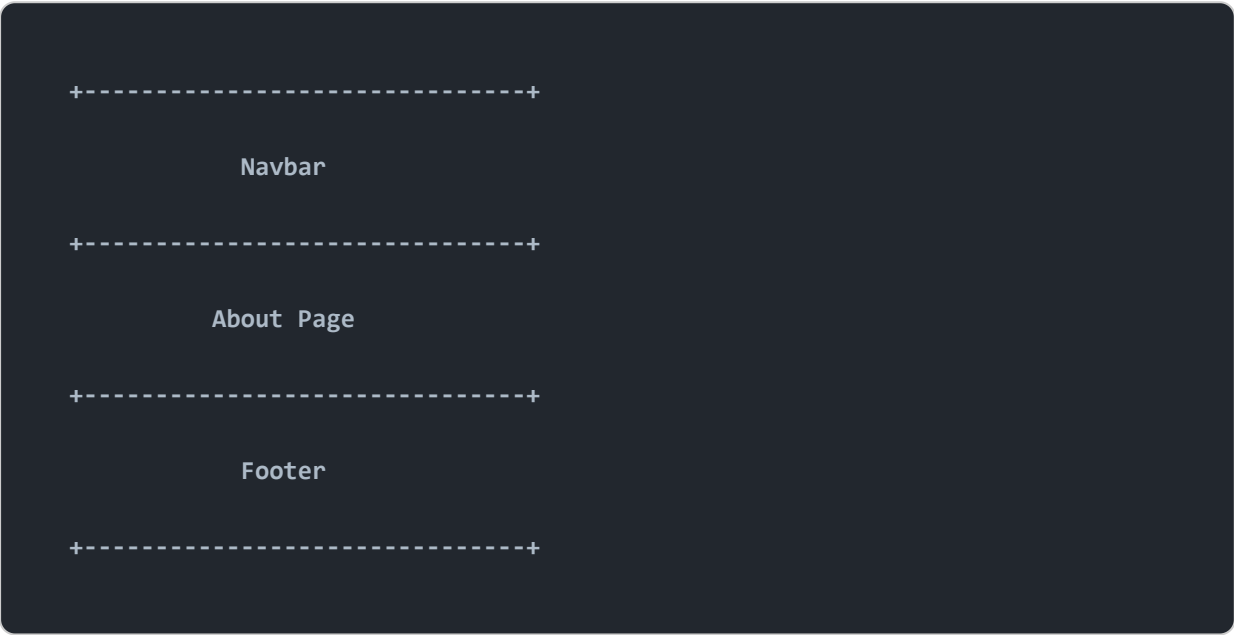


## Home Page

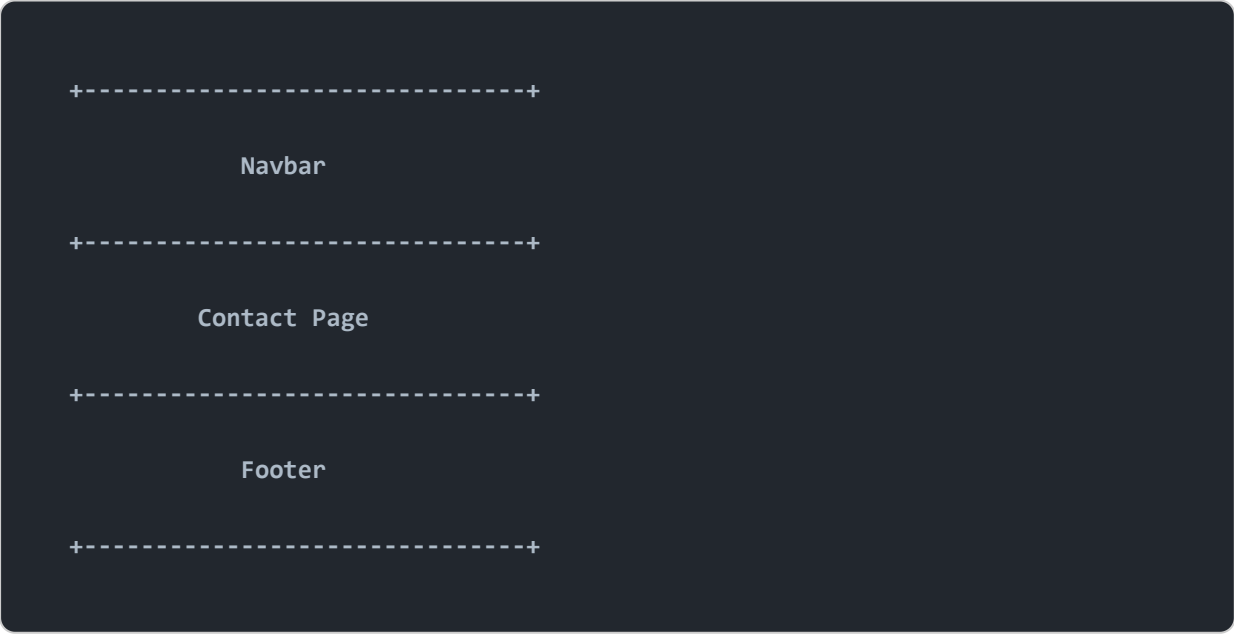




About Page



Contact Page



## Main Screen বলতে কী বোঝায়?

অনেকে "Main Screen" বলতে আসলে **Website-এর মূল Content Area**-কে বোঝায়।

যেখানে প্রতিটি Page-এর Content দেখানো হয়।

উদাহরণ—

Navbar

↓

Main Screen (Outlet)

↓

Footer

এখানে

Main Screen = `<Outlet />`

অর্থাৎ User Home-এ গেলে Home দেখবে, About-এ গেলে About দেখবে, Contact-এ গেলে Contact দেখবে।

## শুধু Navbar Footer নয়

MainLayout-এর ভিতরে আরও অনেক কিছু থাকতে পারে।

```
<>
  <Navbar />

  <Sidebar />

  <Outlet />

  <Newsletter />

  <Footer />
</>
```

## বড় Project-এ MainLayout

---

```
<>
  <ThemeProvider>

    <Navbar />

    <Announcement />

    <ScrollToTop />

    <Outlet />

    <Footer />

  </ThemeProvider>
</>
```

## Dashboard Layout

---

সব Page একই Layout ব্যবহার করে না।

উদাহরণ

Website

Navbar

↓

Outlet

↓

Footer

কিন্তু Dashboard

Sidebar

↓

Topbar

↓

Outlet

তাই Dashboard-এর জন্য আলাদা Layout তৈরি করা হয়।

```
const DashboardLayout = () => {  
  return (  
    <>  
      <Sidebar />  
  
      <Topbar />  
  
      <Outlet />  
    </>  
  )  
}
```

## একটি Project-এ একাধিক Layout থাকতে পারে

MainLayout

|

└─ Home

└─ About

└─ Contact

└─ Blog

DashboardLayout

|

- └─ Dashboard
- └─ Profile
- └─ Users
- └─ Settings

## Outlet না দিলে কী হবে?

যদি লিখেন

```
<>  
  <Navbar />  
  
  <Footer />  
</>
```

এবং Outlet না থাকে,

তাহলে Child Route-এর কোনো Component Screen-এ Render হবে না।

অর্থাৎ

Navbar

Footer

দেখাবে,

কিন্তু

Home

About

Contact



কোনোটাই দেখা যাবে না।

## বাস্তব উদাহরণ

---

ধরুন আপনি YouTube খুললেন।

প্রতিটি Page-এ

- Logo
- Search Bar
- Sidebar

একই থাকে।

কিন্তু

Main Content পরিবর্তন হয়।

YouTube

↓

Logo

↓

Search

↓

Sidebar

↓

Video List

অথবা

Watch Page

অথবা

History

অথবা

React-এ এই Main Content-এর জায়গাটাই `<Outlet />` ।

## Interview Questions

---

### 1. What is MainLayout?

MainLayout হলো একটি Parent Component যেখানে Common UI (Navbar, Footer, Sidebar ইত্যাদি) রাখা হয়।

### 2. What is `<Outlet />` ?

`<Outlet />` হলো React Router-এর একটি Placeholder যেখানে Child Route Render হয়।

### 3. Why do we use MainLayout?

Code Reuse করার জন্য এবং Common Layout একবার লিখে সব Page-এ ব্যবহার করার জন্য।

### 4. What happens if `<Outlet />` is removed?

Child Route-এর কোনো Component Render হবে না।

### 5. Can a React project have multiple layouts?

হ্যাঁ। যেমন—

- MainLayout
- DashboardLayout
- AuthLayout
- AdminLayout

একটি Project-এ একাধিক Layout থাকতে পারে।

## Key Takeaways

---

- **MainLayout** Common UI এক জায়গায় রাখে।
- `<Outlet />` হলো Child Route Render হওয়ার Placeholder।

- Navbar, Footer, Sidebar বারবার লেখার প্রয়োজন হয় না।
- **MainLayout** Code Duplication কমায়।
- একটি Project-এ একাধিক Layout ব্যবহার করা যায়।
- **<Outlet />** না থাকলে Child Page কখনও Screen-এ দেখা যাবে না।